

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Создание информационной системы для поиска и предоставления услуг
в студенческой среде университета ИТМО.

«HelpMore»

Курсовая работа по дисциплине «Информационные системы»

Этап: №2

Научный руководитель:

Тюрин Иван Николаевич

Выполнили:

Покалюхин Илья Игоревич

Лашкул Андрей Владимирович

Группа: Р3310

Санкт-Петербург, 2025 г.

Оглавление

Содержание

Оглавление	2
Задание	3
1. ER-модель	4
2. Даталогическая модель	5
3. Скрипты для создания, удаления базы данных	6
Удаление БД	6
Создание БД	6
Создание таблиц	6
4. Заполнение базы тестовыми данными	10
5. pl/pgsql-функции и процедуры	12
6. Индексы	15
app_user (U001/U002): авторизация и регистрация	15
service (U003–U005, U009): публикация и поиск услуг/запросов	15
response (U006): отклики	16
message (U007): сообщения	16
feedback (U008–U009): отзывы/рейтинги	17
report и bug_report (U010): модерация	17
Вывод	18

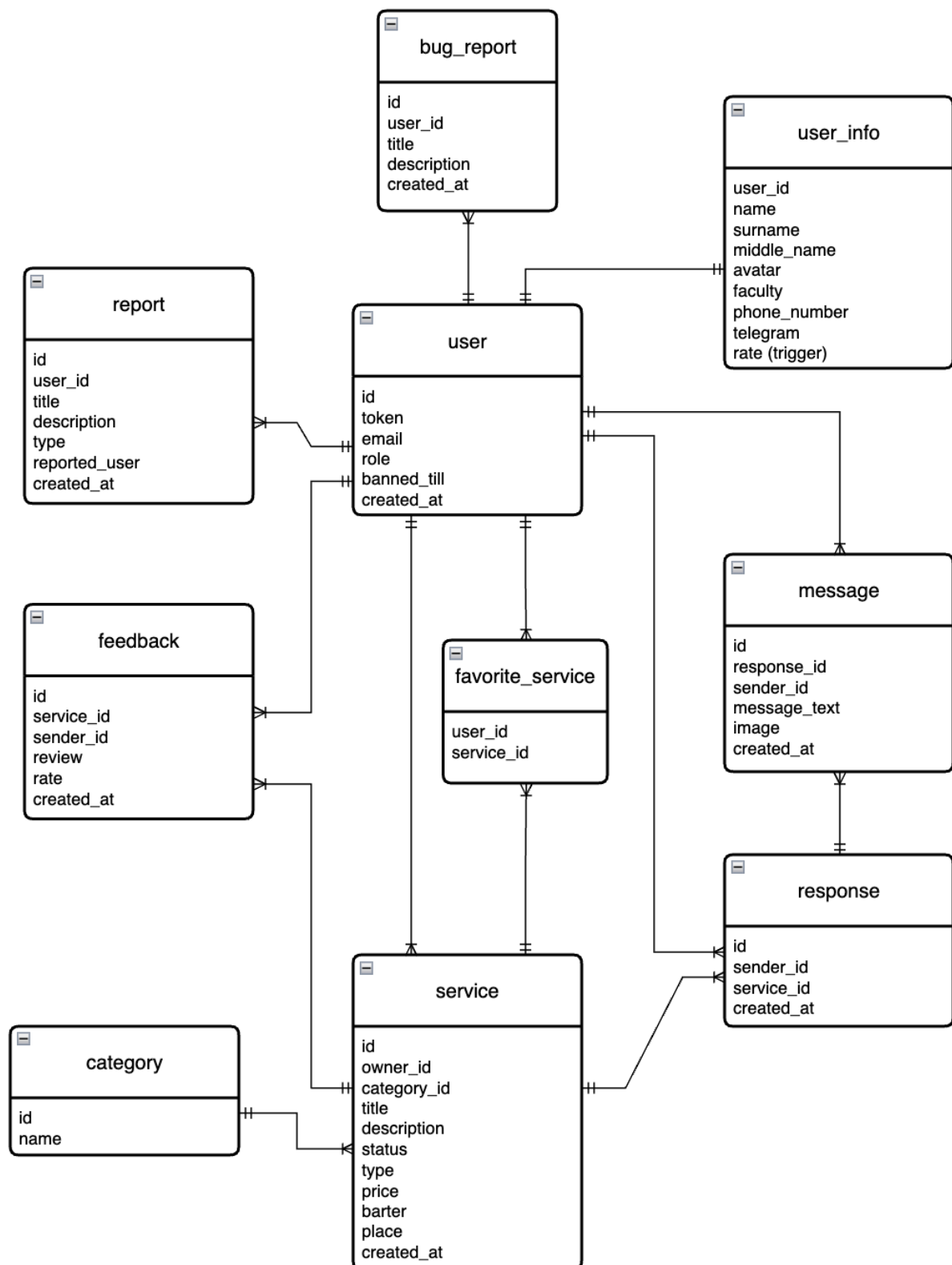
Задание

1. Сформировать ER-модель базы данных (на основе описаний предметной области и прецедентов из предыдущего этапа).

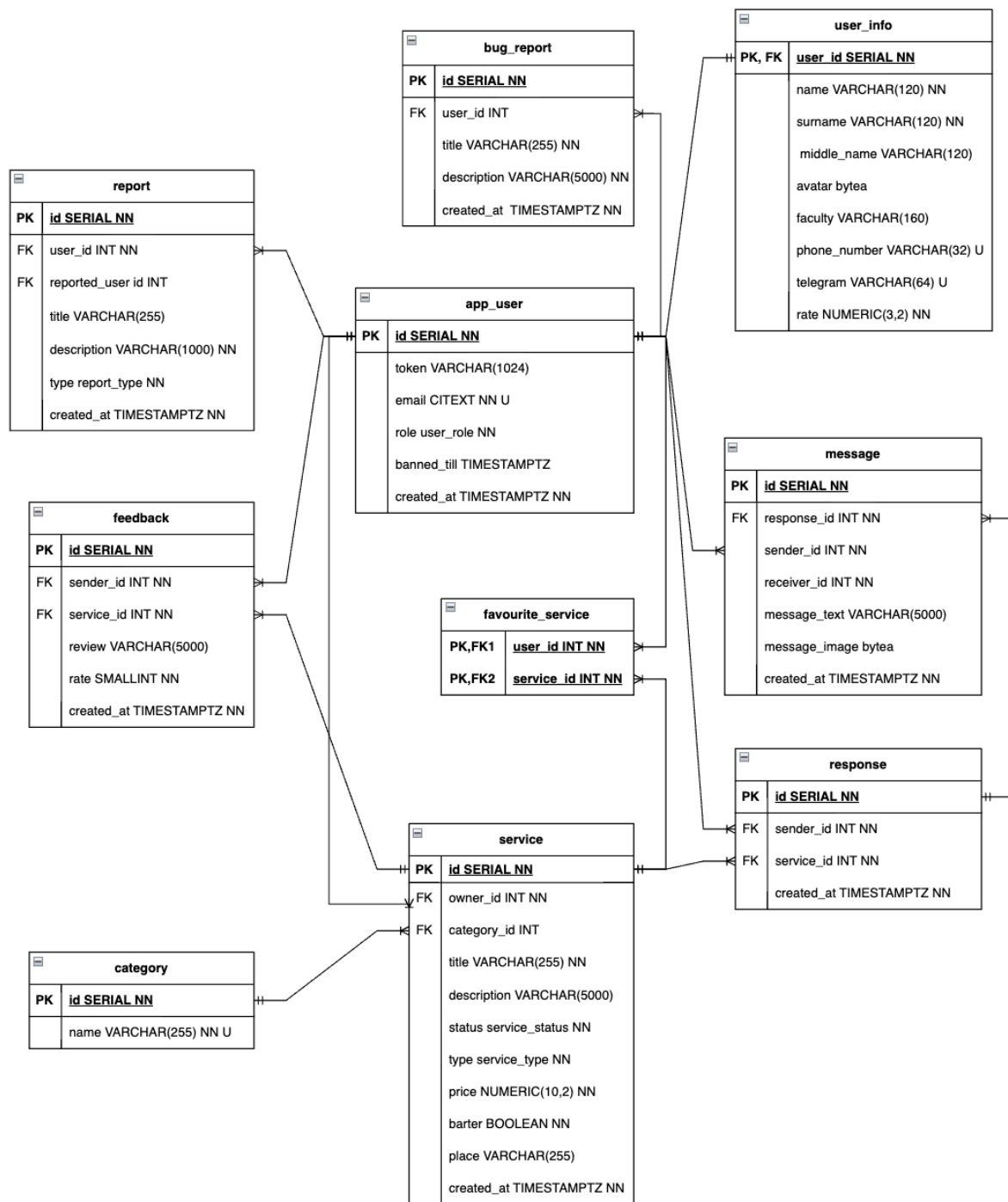
ER-модель должна:

- а. включать в себя не менее 10 сущностей;
 - б. содержать хотя бы одно отношение вида «многие-ко-многим».
2. Согласовать ER-модель с преподавателем. На основе ER-модели построить даталогическую модель.
 3. Реализовать даталогическую модель в реляционной СУБД PostgreSQL.
 4. Обеспечить целостность данных при помощи средств языка DDL и триггеров.
 5. Реализовать скрипты для создания, удаления базы данных, заполнения базы тестовыми данными.
 6. Предложить pl/pgsql-функции и процедуры, для выполнения критически важных запросов (которые потребуются при последующей реализации прецедентов).
 7. Создать индексы на основе анализа использования базы данных в контексте описанных на первом этапе прецедентов. Обосновать полезность созданных индексов для реализации представленных на первом этапе бизнес-процессов.
 8. Составить отчет.

1. ER-модель



2. Даталогическая модель



3. Скрипты для создания, удаления базы данных

Удаление БД

DO \$\$

BEGIN

PERFORM pg_terminate_backend(pid)

FROM pg_stat_activity

WHERE datname = 'uniprofi';

EXCEPTION WHEN others THEN

NULL;

END\$\$;

DROP DATABASE IF EXISTS helpmore;

Создание БД

CREATE DATABASE helpmore

WITH ENCODING 'UTF8'

TEMPLATE template0

LC_COLLATE 'C'

LC_CTYPE 'C';

Создание таблиц

CREATE EXTENSION IF NOT EXISTS citext;

BEGIN;

-- ===== Справочные ENUM'ы =====

DO \$\$

BEGIN

IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'user_role') THEN

CREATE TYPE user_role AS ENUM ('user', 'moderator', 'admin');

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'service_status') THEN

CREATE TYPE service_status AS ENUM ('active', 'archived');

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'service_type') THEN

CREATE TYPE service_type AS ENUM ('offer', 'order');

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'report_type') THEN

CREATE TYPE report_type AS ENUM ('spam', 'fraud', 'insult', 'illegal', 'other');

END IF;

END\$\$;

-- ===== Пользователь =====

```
CREATE TABLE app_user (  
  id          SERIAL PRIMARY KEY,  
  token       VARCHAR(1024) NOT NULL UNIQUE,  
  email       CITEXT NOT NULL UNIQUE,  
  role        user_role NOT NULL DEFAULT 'user',  
  banned_till TIMESTAMPTZ,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE user_info (  
  id          SERIAL PRIMARY KEY  
  user_id     INT REFERENCES app_user(id) ON DELETE CASCADE,  
  name        VARCHAR(120) NOT NULL,  
  surname     VARCHAR(120) NOT NULL,  
  middle_name VARCHAR(120),  
  avatar      BYTEA,  
  faculty     VARCHAR(160),  
  phone_number VARCHAR(32),  
  telegram    VARCHAR(64) UNIQUE,  
  rate        NUMERIC(3,2) NOT NULL DEFAULT 0.00,  
  CONSTRAINT chk_phone_fmt CHECK (phone_number IS NULL OR phone_number ~  
'^[0-9+()\-\s]{5,32}$'),  
  CONSTRAINT chk_rate_bounds CHECK (rate >= 0 AND rate <= 5)  
);
```

-- ===== Категории =====

```
CREATE TABLE category (  
  id          SERIAL PRIMARY KEY,  
  name        VARCHAR(255) NOT NULL UNIQUE  
);
```

-- ===== Услуги =====

```
CREATE TABLE service (  
  id          SERIAL PRIMARY KEY,  
  owner_id    INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,  
  category_id INT NOT NULL REFERENCES category(id),
```

```

title          VARCHAR(255) NOT NULL,
description    VARCHAR(5000) NOT NULL,
status        service_status NOT NULL DEFAULT 'active',
type          service_type  NOT NULL,
price         NUMERIC(10,2)  NOT NULL DEFAULT 0.00,
barter        BOOLEAN        NOT NULL DEFAULT FALSE,
place         VARCHAR(255),
created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT chk_price_nonneg CHECK (price >= 0),
CONSTRAINT unq_service_title_per_owner UNIQUE (owner_id, title)
);

```

-- ===== Избранное =====

```

CREATE TABLE favourite_service (
    user_id    INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    service_id INT NOT NULL REFERENCES service(id)  ON DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, service_id)
);

```

-- ===== Отклики =====

```

CREATE TABLE response (
    id          SERIAL PRIMARY KEY,
    sender_id   INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE, -- кто
откликнулся
    service_id  INT NOT NULL REFERENCES service(id)  ON DELETE CASCADE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT unq_response UNIQUE (sender_id, service_id)
);

```

-- ===== Сообщения по отклику (чат 1-на-1) =====

```

CREATE TABLE message (
    id          SERIAL PRIMARY KEY,
    response_id INT NOT NULL REFERENCES response(id) ON DELETE CASCADE,
    sender_id   INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    receiver_id INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    message_text VARCHAR(5000),
    message_image BYTEA,

```



```

created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT message_not_empty CHECK (
    message_text IS NOT NULL OR message_image IS NOT NULL
)
CONSTRAINT sender_not_receiver CHECK (
    sender_id <> receiver_id
),
);

-- ===== Отзывы =====
CREATE TABLE feedback (
    id          SERIAL PRIMARY KEY,
    sender_id   INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    service_id  INT NOT NULL REFERENCES service(id) ON DELETE CASCADE,
    review      VARCHAR(5000),
    rate        SMALLINT NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_rate_range CHECK (rate BETWEEN 1 AND 5),
    CONSTRAINT unq_feedback_one_per_user UNIQUE (sender_id, service_id)
);

-- ===== Репорты и баг-репорты =====
CREATE TABLE report (
    id          SERIAL PRIMARY KEY,
    user_id     INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    reported_user_id INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    title       VARCHAR(255) NOT NULL,
    description  VARCHAR(1000) NOT NULL,
    type        report_type NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_report_not_self CHECK (user_id <> reported_user_id)
);

CREATE TABLE bug_report (
    id          SERIAL PRIMARY KEY,
    user_id     BIGINT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    title       VARCHAR(255) NOT NULL,
    description  VARCHAR(5000) NOT NULL,

```

```
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
COMMIT;
```

4. Заполнение базы тестовыми данными

```
BEGIN;
```

```
-- ===== Пользователи =====
```

```
INSERT INTO app_user (token, email, role)
```

```
VALUES
```

```
    ('token_user1', 'user1@example.com', 'user'),
    ('token_user2', 'user2@example.com', 'user'),
    ('token_user3', 'user3@example.com', 'user'),
    ('token_admin', 'admin@example.com', 'admin');
```

```
-- ===== Информация о пользователях =====
```

```
INSERT INTO user_info (user_id, name, surname, faculty, telegram)
```

```
VALUES
```

```
    (1, 'Иван', 'Иванов', 'ИТМО', '@ivan_itmo'),
    (2, 'Пётр', 'Петров', 'ИТМО', '@petr_itmo'),
    (3, 'Сергей', 'Сергеев', 'ИТМО', '@serg_itmo'),
    (4, 'Админ', 'Системный', 'ИТМО', '@admin_itmo');
```

```
-- ===== Категории =====
```

```
INSERT INTO category (name)
```

```
VALUES
```

```
    ('Программирование'),
    ('Репетиторство'),
    ('Дизайн'),
    ('Ремонт'),
    ('Моск-собеседование'),
    ('Карьера'),
    ('Другое');
```

```
-- ===== Услуги =====
```

```
-- Иван (user1) предлагает услугу "Написание курсовых"
```

```
INSERT INTO service (owner_id, category_id, title, description, type, price)
```

```
VALUES
```

```
    (1, 1, 'Помощь с курсовыми по Java', 'Вместе разберемся с различными технологиями:
```

```

Spring data, Spring Boot, SQL и другие.', 'offer', 1500.00),
    (1, 2, 'Репетитор по информатике', 'Подготовка к ЕГЭ, ОГЭ, олимпиадам.', 'offer',
1000.00),
    (2, 3, 'Логотипы на заказ', 'Делаю простые и стильные логотипы.', 'offer',
1200.00),
    (2, 1, 'Помощь с курсовой по Python', 'Нужен студент, который поможет разобраться с
pandas и matplotlib.'),
    (3, 4, 'Ремонт ноутбука', 'Перестал включаться ноутбук, требуется диагностика и
ремонт.', 'order', 1000.00),
    (3, 1, 'Настройка Docker проекта', 'Есть проект на Spring + React, нужно помочь с
docker-compose.', 'order', 1337.00);

```

```
-- ===== Отклики =====
```

```
-- Пётр откликается на услугу Ивана
```

```
INSERT INTO response (sender_id, service_id)
```

```
VALUES
```

```
    (2, 1), -- Пётр откликнулся на "Помощь с курсовыми"
```

```
    (3, 1); -- Сергей тоже откликнулся на ту же услугу
```

```
-- ===== Сообщения по отклику =====
```

```
INSERT INTO message (response_id, sender_id, receiver_id, message_text)
```

```
VALUES
```

```
    (1, 2, 1, 'Здравствуйте! Хотел бы обсудить детали.'),
```

```
    (1, 1, 2, 'Конечно, задавайте вопросы.'),
```

```
    (2, 3, 1, 'Здравствуйте, вы делаете проекты по базам данных?');
```

```
-- ===== Отзывы =====
```

```
-- Пётр оставил отзыв Ивану (по первой услуге, 5 звезд)
```

```
INSERT INTO feedback (sender_id, service_id, review, rate)
```

```
VALUES
```

```
    (2, 1, 'Отличная работа, всё вовремя и качественно!', 5);
```

```
-- Сергей оставил отзыв Ивану (по первой услуге, 4 звезды)
```

```
INSERT INTO feedback (sender_id, service_id, review, rate)
```

```
VALUES
```

```
    (3, 1, 'Хорошо, но были задержки.', 4);
```

```
-- теперь rate у Ивана должен обновиться с 5.00 → (5+4)/2 = 4.50

-- ===== Избранное =====
INSERT INTO favourite_service (user_id, service_id)
VALUES
    (2, 1),
    (3, 1),
    (3, 3);

-- ===== Репорты =====
INSERT INTO report (user_id, reported_user_id, title, description, type)
VALUES
    (2, 3, 'Спам', 'Пользователь пишет в личные сообщения без причины', 'spam'),
    (3, 2, 'Оскорбление', 'В отзыве использованы грубые слова', 'insult');

-- ===== Баг-репорт =====
INSERT INTO bug_report (user_id, title, description)
VALUES
    (1, 'Ошибка загрузки аватара', 'Не загружается файл размером более 2 МБ');

COMMIT;
```

5. pl/pgsql-функции и процедуры

```
BEGIN;

-- запрет действий забаненных пользователей — вспомогательная функция
CREATE OR REPLACE FUNCTION assert_not_banned(p_user_id INT)
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE v_until TIMESTAMPTZ;
BEGIN
    SELECT banned_till INTO v_until FROM app_user WHERE id = p_user_id;
    IF v_until IS NOT NULL AND v_until > now() THEN
        RAISE EXCEPTION 'user % is banned until %', p_user_id, v_until
            USING ERRCODE = 'check_violation';
    END IF;
END$$;

-- триггер: запрет публикации услуги для забаненных
CREATE OR REPLACE FUNCTION trg_service_assert_not_banned()
```

```

RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    PERFORM assert_not_banned(NEW.owner_id);
    RETURN NEW;
END$$;

CREATE TRIGGER t_service_not_banned
BEFORE INSERT OR UPDATE ON service
FOR EACH ROW EXECUTE FUNCTION trg_service_assert_not_banned();

-- запрет отклика на свой сервис
CREATE OR REPLACE FUNCTION trg_response_no_self()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_owner INT;
BEGIN
    SELECT owner_id INTO v_owner FROM service WHERE id = NEW.service_id;
    IF v_owner = NEW.sender_id THEN
        RAISE EXCEPTION 'owner cannot respond to own service' USING
ERRCODE='check_violation';
    END IF;
    PERFORM assert_not_banned(NEW.sender_id);
    RETURN NEW;
END$$;

CREATE TRIGGER t_response_no_self
BEFORE INSERT ON response
FOR EACH ROW EXECUTE FUNCTION trg_response_no_self();

-- запрет отзыва на свой сервис
CREATE OR REPLACE FUNCTION trg_feedback_no_self()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_owner INT;
BEGIN
    SELECT owner_id INTO v_owner FROM service WHERE id = NEW.service_id;
    IF v_owner = NEW.sender_id THEN
        RAISE EXCEPTION 'owner cannot leave feedback to own service' USING
ERRCODE='check_violation';
    END IF;

```

```

    PERFORM assert_not_banned(NEW.sender_id);
    RETURN NEW;
END$$;

CREATE TRIGGER t_feedback_no_self
BEFORE INSERT ON feedback
FOR EACH ROW EXECUTE FUNCTION trg_feedback_no_self();

-- поддержка агрегированного рейтинга исполнителя (owner) в user_info.rate
CREATE OR REPLACE FUNCTION recalc_owner_rate(p_owner_id INT)
RETURNS VOID LANGUAGE plpgsql AS $$
BEGIN
    UPDATE user_info ui
    SET rate = COALESCE((
        SELECT ROUND(AVG(f.rate)::numeric, 2)
        FROM feedback f
        JOIN service s ON s.id = f.service_id
        WHERE s.owner_id = p_owner_id
    ), 0)
    WHERE ui.user_id = p_owner_id;
END$$;

CREATE OR REPLACE FUNCTION trg_feedback_recalc_rate()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_owner INT;
BEGIN
    SELECT owner_id INTO v_owner FROM service WHERE id =
COALESCE(NEW.service_id, OLD.service_id);
    PERFORM recalc_owner_rate(v_owner);
    RETURN COALESCE(NEW, OLD);
END$$;

CREATE TRIGGER t_feedback_recalc_ins
AFTER INSERT ON feedback
FOR EACH ROW EXECUTE FUNCTION trg_feedback_recalc_rate();

CREATE TRIGGER t_feedback_recalc_upd

```

```
AFTER UPDATE ON feedback
FOR EACH ROW EXECUTE FUNCTION trg_feedback_recalc_rate();
```

```
CREATE TRIGGER t_feedback_recalc_del
AFTER DELETE ON feedback
FOR EACH ROW EXECUTE FUNCTION trg_feedback_recalc_rate();
```

6. Индексы

app_user (U001/U002): авторизация и регистрация

Быстрые проверки бана и поиск по токену.

```
CREATE INDEX IF NOT EXISTS app_user_banned_active_idx
ON app_user (banned_till)
WHERE banned_till IS NOT NULL;
```

```
CREATE INDEX IF NOT EXISTS idx_app_user_token
ON app_user (token);
```

Индексы необходимы для выполнения требований:

- F1.1, F2.16 — вход/выход через OAuth (Google / Яндекс) и работа с сессиями.
- F2.17, F3.3, F4.3 — проверка, не заблокирован ли пользователь (при логине), а также блокировка и разблокировка пользователей.

Оба индекса btree, потому что он наилучшим подходит для операций =, <, >

service (U003–U005, U009): публикация и поиск услуг/запросов

Для поиска услуг по фильтрам

```
CREATE INDEX IF NOT EXISTS service_pub_cat_type_created_idx
ON service (category_id, type, created_at DESC)
WHERE status = 'active';
```

Для поиска всех активных услуг

```
CREATE INDEX IF NOT EXISTS service_pub_created_idx
ON service (created_at DESC)
WHERE status = 'active';
```

Для просмотра своих услуг

```
CREATE INDEX IF NOT EXISTS service_owner_created_idx
ON service (owner_id, created_at DESC);
```

Для поиска в ценовом диапазоне

```
CREATE INDEX IF NOT EXISTS idx_service_price
ON service (price);
```

Для поиска по названию услуги (title)

```
CREATE INDEX idx_service_title_trgm ON service USING gin (title gin_trgm_ops);
```

Индексы необходимы для выполнения требований:

- F2.1 — просмотр открытых предложений и заказов (лента активных услуг).
- F2.4, F2.5 — публикация заказов и предложений (после вставки нужно быстро видеть их в списке)
- F2.6, F2.7 — поиск и фильтрация по категории и стоимости, и текстовый поиск по названию.
- F2.13 — личная история заказов/предложений (фильтр по owner_id)
- F2.14 — избранное.
- F3.1 — модератору проще находить и удалять проблемные заказы/предложения.

service_pub_cat_type_created_idx, service_pub_created_idx, service_owner_created_idx,

idx_service_price — btree: так как используют точное сравнение

service_pub_* - частичные: так как почти всегда работа ведется с активными услугами

idx_service_title_trgm — GIN + pg_trgm: для поиска по подстроке/опечаткам

response (U006): отклики

Отклики по услуге, с порядком по времени

```
CREATE INDEX IF NOT EXISTS idx_response_service_created_at  
ON response (service_id, created_at DESC);
```

Отклики, отправленные пользователем

```
CREATE INDEX IF NOT EXISTS idx_response_sender_created_at  
ON response (sender_id, created_at DESC);
```

Индексы необходимы для выполнения требований:

- F2.8, F2.9 — отклик на заказы и предложения:
 - быстрый показ списка откликов на конкретный заказ/услугу.
- F2.13 — история своих откликов.
- F2.10 — старт чата строится от отклика.

Оба индекса btree, потому что он наилучшим подходит для операций =, <, >

message (U007): сообщения

Сообщения по конкретному отклику

```
CREATE INDEX IF NOT EXISTS idx_message_response_id  
ON message (response_id);
```

Входящие/исходящие по пользователю с датой

```
CREATE INDEX IF NOT EXISTS idx_message_receiver_created_at  
ON message (receiver_id, created_at);
```

```
CREATE INDEX IF NOT EXISTS idx_message_sender_created_at
```



```
ON message (sender_id, created_at);
```

```
CREATE INDEX IF NOT EXISTS message_pair_created_idx
```

```
ON message (  
    LEAST(sender_id, receiver_id),  
    GREATEST(sender_id, receiver_id),  
    created_at DESC  
);
```

Индексы необходимы для выполнения требований:

- F2.10 — чат между пользователями:
 - загрузка истории диалога
 - список входящих/исходящих сообщений.
- F2.8, F2.9 — связи по откликам (чат по конкретному отклику).

Все индексы btree, потому что они наилучшим подходит для операций =, <, >

feedback (U008–U009): отзывы/рейтинги

Отзывы по услуге, свежие сверху

```
CREATE INDEX IF NOT EXISTS idx_feedback_service_created_at
```

```
ON feedback (service_id, created_at DESC);
```

Отзывы, оставленные пользователем

```
CREATE INDEX IF NOT EXISTS idx_feedback_sender_created_at
```

```
ON feedback (sender_id, created_at DESC);
```

Индексы необходимы для выполнения требований:

- F2.11 — оставление оценок и отзывов.
- F2.2, F2.12 — просмотр рейтинга и отзывов в профиле/на услуге.
- F2.13 — история оставленных отзывов.

Все индексы btree, потому что они наилучшим подходит для операций =, <, >

report и bug_report (U010): модерация

```
CREATE INDEX IF NOT EXISTS report_reported_created_idx
```

```
ON report (reported_user_id, created_at DESC);
```

```
CREATE INDEX IF NOT EXISTS report_type_idx
```

```
ON report (type);
```

```
CREATE INDEX IF NOT EXISTS bug_report_created_idx
```

```
ON bug_report (created_at DESC);
```

```
CREATE INDEX IF NOT EXISTS bug_report_user_idx  
ON bug_report (user_id);
```

- F2.15 — подача жалоб пользователями:
 - модератор быстро видит жалобы на конкретного пользователя.
- F3.1 — удаление некорректных заказов/предложений.
- F3.2 — рассмотрение жалоб пользователями (очередь жалоб по дате и по типам).
- F3.3 — блокировка нарушителей (быстрая выборка всех жалоб на пользователя).

Все индексы btree, потому что они наилучшим подходит для операций =, <, >

Вывод

В результате выполнения второго этапа курсовой работы по дисциплине "Информационные системы", была разработана структура реляционной базы данных на базе заранее сформулированных ER- и даталогической моделей будущей информационной системы. Полученная структура БД соответствует заданным требованиям и обеспечивает нужный уровень целостности данных. В дальнейшем при необходимости система может быть переработана с учётом новых требований.